

Sprzętowa implementacja funkcji aktywacji ReLU, sigmoid i tanh

Jakub Jucha Patryk Kowalczyk

Czerwiec 2026

Spis treści

1	Wprowadzenie	3
2	Skrótowa lista potencjalnych metod	5
2.1	ReLU	5
2.2	Sigmoid i tanh	5
3	Implementacja	6
3.1	ReLU	6
3.2	tanh	6
4	Porównywanie metod	9
4.1	Planowane środowisko symulacyjne i model referencyjny	9
4.2	Kryteria oceny	9
4.2.1	Dokładność (Analiza błędu)	9
4.2.2	Zużycie zasobów FPGA	10
4.2.3	Szybkość działania	10

1 Wprowadzenie

Główną zaletą procesorów (CPU) jest ich uniwersalność — są one w stanie obsłużyć dowolną funkcję matematyczną. Jednak w systemach wbudowanych i akceleratorach sprzętowych stosuje się inne techniki. Funkcja aktywacji utrwalona bezpośrednio w strukturze krzemowej może zwracać wynik już po jednym cyklu zegara, co czyni takie podejście znacznie korzystniejszym pod względem efektywności energetycznej. Istotną wadą tego rozwiązania jest konieczność wydzielania dedykowanej powierzchni na krzemie oraz brak możliwości późniejszej modyfikacji za pomocą zwykłej aktualizacji oprogramowania [3]. Warto jednak zauważyć, że w przypadku układów FPGA ten problem elastyczności nie występuje.

Najprostszą do zaimplementowania w języku Verilog jest funkcja ReLU (ang. *Rectified Linear Unit*), opisana wzorem $y = \max(0, x)$. Jej realizacja sprzętowa wymaga jedynie komparatora znaku oraz prostego multipleksera: gdy sygnał wejściowy jest ujemny, układ zwraca 0, natomiast w przeciwnym wypadku sygnał wejściowy jest przepuszczany bezpośrednio na wyjście [3].

Kolejną popularną metodą jest przybliżanie wartości funkcji za pomocą tablic przeglądowych LUT (ang. *Look Up Table*), czyli poprzez zapisanie predefiniowanych wartości wejść i wyjść w pamięci. Zaletą tego podejścia jest wysoka szybkość działania oraz możliwość szybkiej podmiany zawartości tablicy na inną. Wadą jest natomiast fakt, że przy wymaganej wyższej precyzji obliczeń rozmiar tablicy drastycznie rośnie [3].

Pewnym ulepszeniem tej koncepcji są tablice LUT z interpolacją. Metoda ta również opiera się na strukturze tablicowej, ale wykorzystuje dodatkowe operacje arytmetyczne do estymacji wyników pomiędzy węzłami tablicy, co pozwala poprawić dokładność bez nadmiernego zwiększania zużycia pamięci. Istnieją również architektury wykorzystujące dwie tablice: jedną o grubszej, a drugą o drobniejszej ziarnistości, których wyniki są sumowane w celu uzyskania ostatecznego przybliżenia [1].

W aplikacjach cyfrowego przetwarzania sygnałów (DSP), które wymagają wyznaczania funkcji nieliniowych, takich jak tangens hiperboliczny ($\tanh$), szeroko stosowany jest algorytm CORDIC (ang. *Coordinate Rotation Digital Computer*). Jest to niezwykle elegancka metoda iteracyjna, która wykorzystuje wyłącznie prostą logikę przesunięć bitowych oraz dodawania (ang. *shift-add*) do osiągnięcia bardzo dokładnych wyników. Przykładowa implementacja CORDIC dla funkcji sinus i cosinus została przedstawiona pod adresem <https://zipcpu.com/dsp/2017/08/30/cordic.html>. Większość

wiodących dostawców układów FPGA oferuje również gotowe bloki własności intelektualnej (ang. *IP cores*) dla jednostek CORDIC [1].

Innym podejściem są szeregi Taylora (ang. *Taylor series*) oraz przybliżenia wielomianowe (ang. *polynomial approximations*). Metody te wykorzystują tradycyjne rozwinięcia funkcji w szeregi potęgowe i próbują zaimplementować operacje na zmiennych wyższych rzędów w wielomianach, co jednak wiąże się z koniecznością stosowania zasobożernych układów mnożących [1].

Do najbardziej zaawansowanych rozwiązań należą metody oparte na interpolacji wykorzystującej dyskretną transformację cosinusową DCT (ang. *Discrete Cosine Transform*). Pozwalają one na bardzo precyzyjne dopasowanie dokładności obliczeń przy jednoczesnej optymalizacji zasobów sprzętowych wymaganych dla danej aplikacji [1]. Przykładowa implementacja funkcji $\tanh$ z wykorzystaniem tej techniki została szczegółowo opisana w artykule z 2016 roku [2].

Aproksymacja odcinkowa (PWL - Piecewise Linear Approximation) jest kolejną metodą, która polega na podziale dziedziny funkcji na segmenty i aproksymacji każdego z nich za pomocą prostej linii [4]. W sprzęcie implementuje się to za pomocą małej tablicy LUT (która na podstawie wartości wejścia wybiera odpowiedni przedział, czyli współczynniki a i b) oraz jednego mnożnika i dodawacza. W zaawansowanych implementacjach (np. algorytm PLAN) dobiera się współczynniki a tak, aby były potęgami dwójki (2^{-k}) - wtedy mnożnik zastępuje się zwykłym przesunięciem bitowym, co drastycznie oszczędza zasoby FPGA.

2 Skrótowa lista potencjalnych metod

2.1 ReLU

- Prosta implementacja w Verilog'u opisana poniżej

2.2 Sigmoid i tanh

- Implementacja za pomocą LUT
- Implementacja za pomocą LUT z interpolacją
- Implementacja za pomocą CORDIC
- Implementacja za pomocą Taylor series i polynomial approximations
- Implementacja za pomocą DCT Interpolation
- Implementacja za pomocą aproksymacji odcinkowej (PWL) + dodatkowo może być ulepszona o dobór współczynników a tak, aby były potęgami dwójki (PLAN)

3 Implementacja

Poniższe implementacje zostały znalezione na internecie i nie zostały jeszcze przetestowane.

3.1 ReLU

```
1 module relu (  
2     input wire signed [31:0] din_relu,  
3     output wire signed [31:0] dout_relu  
4 );  
5     assign dout_relu = (din_relu < 0) ? 32'b0 : din_relu;  
6 endmodule
```

Listing 3.1: Implementacja funkcji ReLU w Verilog'u [1]

3.2 tanh

```
1 module tanh_lut #(
2     parameter AW = 10,
3     parameter DW = 16,
4     parameter N = 16,
5     parameter Q = 12
6 ) (
7     input clk,
8     input [AW-1:0] phase,
9     output [DW-1:0] tanh
10 );
11     reg [AW-1:0] addr_reg;
12 (* ram_style = "block" *) reg [DW-1:0] mem [1<<AW-1:0];
13 initial
14 begin
15     $readmemb("tanh_data.mem", mem);
16 end
17
18 always@(posedge clk)
19 begin
20     addr_reg <= phase[AW-1:0];
21 end
22
23 assign tanh = mem[addr_reg];
```

Listing 3.2: Implementacja funkcji tanh w Verilog'u używając LUT [1]

```

1  module tanh_lut #(
2      parameter AW = 10,
3      parameter DW = 16,
4      parameter N = 16,
5      parameter Q = 12
6  ) (
7      input clk,
8      input [N-1:0] phase,
9      output [DW-1:0] tanh
10 );
11
12     reg [9:0] addra_reg;
13     reg [9:0] addrb_reg;
14     wire [15:0] tanha;
15     wire [15:0] tanhb;
16     wire ovr1, ovr2;
17
18     wire [15:0] frac, one_minus_frac;
19     wire [15:0] A1, A2;
20     wire [15:0] one;
21     wire [DW-1:0] tanh_temp;
22
23
24     (* ram_style = "block" *) reg [15:0] mem [1<<10-1:0];
25     initial
26     begin
27         $readmemb("tanh_data.mem", mem);
28     end
29     always@(posedge clk)
30     begin
31         addra_reg <= phase[9:0];
32         addrb_reg <= phase[9:0] + 1'b1;
33     end
34     assign tanha = mem[addra_reg];
35     assign tanhb = mem[addrb_reg];
36     assign frac = {'d0, phase[N-AW-'d2-1:0]};
37     assign one = 16'b0001000000000000;
38     assign one_minus_frac = one - frac;
39     qmult #(N,Q) mul1 (tanha, frac, A1, ovr1);
40     qmult #(N,Q) mul2 (tanhb, one_minus_frac, A2, ovr2);
41     assign tanh_temp = A1 + A2;
42     assign tanh = (phase [N-1]) ? (phase[N-2] ? (16'b1111000000000000) :
43
44 endmodule

```

Listing 3.3: Implementacja funkcji tanh w Verilog'u używając LUT z interpolacją [1]

4 Porównywanie metod

Opis w jaki sposób zamierzamy porównać ze sobą wszystkie opisane wcześniej metody implementacji.

4.1 Planowane środowisko symulacyjne i model referencyjny

Wszystkie moduły sprzętowe zostaną napisane w języku Verilog. Do ich uruchomienia, symulacji oraz stworzenia środowisk testowych (ang. *testbench*) planujemy wykorzystać darmową platformę internetową EDA Playground (<https://www.edaplayground.com>).

Działanie testbencha będzie polegać na automatycznym podawaniu na wejście układu RTL całego zakresu zmiennych i zapisywaniu tego, co układ zwróci na wyjściu. Żeby sprawdzić jakość działania, wyniki z Veriloga będą porównywane z idealną teorią matematyczną. Jako idealny wzorzec (referencję) zamierzamy wykorzystać obliczenia wykonane w Pythonie na liczbach zmiennoprzecinkowych.

4.2 Kryteria oceny

Każdą z metod będę oceniać pod kątem trzech głównych parametrów: dokładności, zużycia zasobów oraz szybkości działania.

4.2.1 Dokładność (Analiza błędu)

Do zmierzenia błędu planujemy wykorzystać trzy metryki:

- **Błąd maksymalny** – pokaże największą różnicę między teorią a Verilogiem, jaka przytrafi się w całym teście. Wskaże to na „najgorszy możliwy przypadek”.
- **Średni błąd bezwzględny (MAE)** – pokaże, o ile średnio myli się projektowany układ dla wszystkich podanych próbek.
- **Pierwiastek błędu średniokwadratowego (RMSE)** – podobnie jak MAE pokaże błąd, ale znacznie mocniej ukarze duże, pojedyncze odchylenia od wykresu.

4.2.2 Zużycie zasobów FPGA

Z raportów po syntezie układu będą odczytywane dane o tym, ile miejsca w strukturze FPGA zajmie dana metoda. Sprawdzane będą trzy kluczowe elementy:

- **LUT** – uniwersalne komórki logiczne. Ich liczba powie o tym, jak skomplikowana logicznie jest dana metoda.
- **FF (Przerzutniki)** – elementy pamięciowe.
- **DSP** – wbudowane w FPGA szybkie sprzętowe mnożniki. Ważnym celem projektowym będzie ich oszczędzanie, ponieważ ich liczba w układach FPGA jest mocno ograniczona.

4.2.3 Szybkość działania

Wydajność czasowa układów zostanie sprawdzona na podstawie dwóch parametrów:

- **Maksymalna częstotliwość (F_{max})** – będzie informować o tym, z jak szybkim zegarem (w MHz) może maksymalnie pracować dany moduł, żeby nie doszło do błędów.
- **Opóźnienie (ang. *Latency*)** – czyli liczba cykli zegara, jaką układ będzie potrzebował od momentu wrzucenia liczby na wejście do momentu, gdy prawidłowy wynik pojawi się na wyjściu.

Bibliografia

- [1] *Activation Function*. URL: <https://thedatabus.in/activation-function/> (term. wiz. 03.06.2026).
- [2] Ahmed M. Abdelsalam et al. „Accurate and Efficient Hyperbolic Tangent Activation Function on FPGA using the DCT Interpolation Filter”. W: (2016). URL: <https://arxiv.org/pdf/1609.07750v1>.
- [3] Bryon Moyer. *Implementing AI Activation Functions*. URL: <https://semiengineering.com/implementing-ai-activation-functions/> (term. wiz. 03.06.2026).
- [4] *PWL Model*. URL: https://www.bohrium.com/en/sciencepedia/feynman/keyword/pwl_model (term. wiz. 03.06.2026).